

DATA ANALYTICS AND DATA VISUALIZATION

MINI PROJECT ON

FLIGHT FARE ANALYSIS AND PREDICTION

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE & ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with A+)

Sangivalasa, Bheemili Mandal, Visakhapatnam - 531162. (A.P)

2026-2027

BONAFIDE CERTIFICATE

This is to certify that the mini project report entitled “**FLIGHT FARE ANALYSIS AND PREDICTION**” submitted as part of the curriculum for the **Data Analytics and Data Visualization** course by the **II/IV Computer Science and Engineering** student at **Anil Neerukonda Institute of Technology & Sciences (ANITS), Visakhapatnam**, is a record of bona fide work carried out under my supervision.

SUBMITTED BY

S. KOTESWARAO - (A24126510107)

Faculty-In charge

Mrs. Y. Padma sri
Assistant Professor

Head of Department - CSE

Dr. G. Srinivas
Head of Department - CSE, ANITS

FLIGHT FARE (DAV) ANALYSIS AND PREDICTION

Data Analytics and Visualization (DAV) Mini Project Documentation

Project Type: Flight Fare Data Analytics, Machine Learning Prediction and Streamlit Web Application

Programming Language: Python

Libraries / Technologies: Pandas, NumPy, Scikit-learn, Plotly, Streamlit

Student: S. Koteswarao (A24126510107), II/IV B.Tech CSE, ANITS

Dataset: 10,000 domestic flight booking records across 6 metro Indian cities with 19 columns; all 10,000 records remain after cleaning

GitHub Repository: <https://github.com/koteswaraosabbavarapu-stack/flight-fare-analytics>

Project Goal: Analyse which factors drive ticket prices on Indian domestic routes, study non-linear pricing effects (last-minute booking surge, seat-demand pricing, travel class multipliers), compare Linear Regression with Random Forest and build an interactive web application that predicts the fare of a flight booking.

1. Introduction

Data Analytics and Visualization is the process of collecting, cleaning, exploring and presenting data so that useful patterns can be understood easily. Airline ticket prices are dynamic: the same seat can cost very different amounts depending on demand, how early it is booked, the distance flown, the airline and the season. For travellers, understanding these drivers helps in booking at the right time; for airlines, fare prediction supports revenue management and seat allocation.

This project applies the DAV workflow to a dataset of 10,000 domestic flight bookings between six metro cities and extends it with machine learning to predict the fare. It covers data loading, cleaning, exploratory data analysis, visualization, training and comparison of two regression models, and a multi-page Streamlit web application that presents the analysis and gives live fare predictions.

2. Objectives

- Clean and prepare a dataset that contains missing values and verify that there are no duplicate records.
- Find how airline, travel class, booking window, seat availability and departure slot influence ticket prices.
- Quantify the steep non-linear price surge for last-minute bookings and for flights with few seats left.
- Create clear, interactive visualizations that communicate the main patterns in the data.
- Implement and compare two machine learning models that predict the fare (fare_inr) using R^2 , RMSE and MAE.
- Deliver the results through a web interface with a live fare prediction form.

3. Tools and Technologies Used

Python 3.14: Main programming language for data preparation, analysis, modelling and the web server.

Pandas: DataFrames, grouping, aggregation, filtering, median imputation and duplicate removal.

NumPy: Numerical operations, vectorised calculations and exponential / logarithmic functions.

Scikit-learn: ColumnTransformer, SimpleImputer, StandardScaler, OneHotEncoder, Pipeline, Linear Regression, Random Forest and the evaluation metrics (R^2 , RMSE, MAE).

Plotly Express and Graph Objects: Interactive charts with hover, zoom and colour maps shown inside the web application.

Matplotlib and Seaborn: Static charts used in this report.

Streamlit: Web framework used to build the five-page interactive dashboard.

Git and GitHub: Version control and hosting of the source code.

4. Project Workflow

Raw synthetic data (10,000 records) → Pandas DataFrame → Data cleaning (median imputation) → Exploratory analysis → Plotly visualization → Preprocessing pipeline (imputer + scaler + one-hot encoder) → 80/20 train/test split → Model training and comparison → Streamlit web application → Live fare prediction → Deployment on Streamlit Community Cloud and GitHub

The dataset generator is kept in data.py, while the cleaning, modelling and dashboard logic is kept in app.py, so the same prepared data feeds the charts and the models.

5. Important Code Used

```
import numpy as np
import pandas as pd
SEED = 2026
N_RECORDS = 10_000
# Base fare calculation with realistic non-linear multipliers
base = 1800 + 3.2 * distance
advance_mult = 1 + 1.6 * np.exp(-days_before / 8) # last-minute spike
demand_mult = 1 + 0.5 * ((100 - seats) / 100) ** 2 # fewer seats -> pricier
stops_mult = 1 - 0.07 * stops
fare = (
    base
    * np.array([AIRLINE_PRICE_MULT[a] for a in airline])
    * np.array([CLASS_MULT[c] for c in travel_class])
    * advance_mult * demand_mult * stops_mult
    * np.array([TIME_MULT[t] for t in departure])
    * np.array([DAY_MULT[d] for d in day])
    * np.where(holiday == "Yes", 1.18, 1.0)
)
fare += (baggage - 15) * 45 + np.where(meal == "Yes", 350, 0)
fare += np.array([CHANNEL_FEE[c] for c in channel])
fare *= rng.lognormal(0, 0.06, n)
```

The dataset is generated with a fixed seed, so the same 10,000 records are recreated every time. The fare starts from a base price that grows with distance and is then multiplied by airline, travel class, booking-window, seat-demand, stops, departure-slot, weekday and holiday factors. The booking-window factor $1 + 1.6e^{-\text{days}/8}$ makes prices rise sharply in the last days before departure, and the seat factor makes nearly full flights more expensive. Baggage, meal and booking-channel add-ons and a small random noise are added at the end.

Data cleaning and imputation (app.py)

```
@st.cache_data
def clean_data(raw: pd.DataFrame):
    df = raw.copy()
    report = []
    for col in ["duration_hours", "seat_availability_pct", "airline_rating"]:
        n_missing = int(df[col].isna().sum())
        median = float(df[col].median())
        df[col] = df[col].fillna(median)
        report.append({"Column": col, "Missing values": n_missing,
                       "Cleaning method": f"Filled with median ({median:.2f})"})
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DATA ANALYTICS AND DATA VISUALIZATION LAB

```
n_dupes = int(df.duplicated().sum())
df = df.drop_duplicates()
return df, pd.DataFrame(report), n_dupes
```

Each of the three columns with gaps is filled with its own median, which is robust to the long tail of extreme values, and the table is then checked for duplicate rows. No rows are deleted, so all 10,000 records are retained.

Preprocessing pipeline and model training (app.py)

```
def make_pre():
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc", StandardScaler())]), num_cols),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                          ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
    ])
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=150, random_state=42, n_jobs=-1),
}
```

Numeric features are median-imputed and standardised, categorical features are filled with the most frequent value and one-hot encoded, and both models use exactly the same preprocessing and the same 80% / 20% split (8,000 training and 2,000 test records).

6. Dataset Description

The dataset contains **10,000 flight records in 19 columns** for domestic bookings between six metro cities: Delhi, Mumbai, Bengaluru, Hyderabad, Kolkata and Chennai. It is a reproducible synthetic dataset, and missing values were injected on purpose so that the cleaning step can be demonstrated.

Column	Type	Description
flight_id	String	Unique booking identifier (e.g. FL100001)
airline	Categorical	IndiGo, Air India, Akasa Air, SpiceJet, Air India Express, Alliance Air
source_city	Categorical	Origin metro city
destination_city	Categorical	Destination metro city
distance_km	Numeric	Route distance, 290 km to 1,760 km
departure_time	Categorical	Early Morning, Morning, Afternoon, Evening, Night
day_of_week	Categorical	Day of travel, Monday to Sunday
stops	Numeric	Number of layovers (0, 1 or 2)
travel_class	Categorical	Economy, Premium Economy, Business
days_before_departure	Numeric	Booking lead time, 1 to 60 days
duration_hours	Numeric	Flight travel time in hours
seat_availability_pct	Numeric	Unsold seats at booking time (3% to 100%)
baggage_kg	Numeric	Included baggage allowance (15, 20, 25, 30 kg)
meal_included	Categorical	In-flight meal included (Yes / No)
airline_rating	Numeric	Customer rating (1.0 to 5.0)
booking_channel	Categorical	Website, Mobile App, Travel Agent

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DATA ANALYTICS AND DATA VISUALIZATION LAB

Column	Type	Description
holiday_season	Categorical	Peak holiday period (Yes / No)
fare_inr	Numeric (target)	Final ticket fare in Indian Rupees (prediction target)
booking_status	Categorical	Confirmed, Rescheduled, Cancelled

7. Data Cleaning and Preparation

Data preparation turns the raw table into a reliable modelling table. The following steps were applied and measured:

Step	Result
Duplicate rows removed	0 rows (verified with drop_duplicates)
duration_hours missing values	300 (3.0%) filled with the median, 2.69 h
seat_availability_pct missing values	200 (2.0%) filled with the median, 54.9%
airline_rating missing values	250 (2.5%) filled with the median, 3.9
Total missing cells imputed	750 cells
Numeric model features standardised (mean 0, SD 1)	7 features
Categorical model features one-hot encoded	9 features
Columns excluded from model inputs	flight_id, fare_inr (target), booking_status
Final clean dataset	10,000 rows (100% retained)

The target is the numeric fare_inr. Linear Regression uses the standardised features inside a scikit-learn Pipeline. The data is split randomly: the training set has 8,000 records (80%) and the test set has 2,000 records (20%).

8. Exploratory Data Analysis (EDA)

Exploratory Data Analysis examines the structure, distributions and relationships in the data before modelling. In this project EDA uses Pandas aggregations together with visualization.

```
df.shape
df.isna().sum()
df['booking_status'].value_counts(normalize=True)
df.groupby('travel_class')['fare_inr'].mean()
df.groupby('airline')['fare_inr'].mean().sort_values()
df.groupby(pd.cut(df['days_before_departure'], [0, 3, 7, 14, 30, 60]))['fare_inr'].mean()
df[numeric_cols + ['fare_inr']].corr()
```

These operations summarise 10,000 bookings into a few meaningful numbers, such as the average fare of each travel class, airline or booking window.

9. Data Visualization Techniques

The project uses a horizontal bar chart for missing values, a donut chart for booking status, a histogram with density curve for the fare distribution, bar charts for travel class, airline and departure-slot comparison, a scatter plot coloured by travel class for booking window versus fare, a line chart for fare versus route distance, a box plot for the spread of fares by booking window, a correlation heatmap, an actual-versus-predicted scatter plot and feature-importance bars for model evaluation. The web application shows the same analysis with interactive Plotly charts.

10. Project Output – Data Quality and Fare Patterns

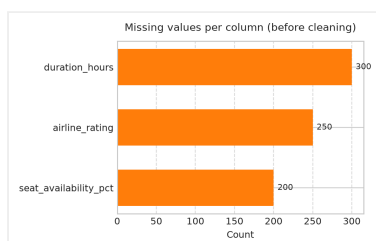


Figure 1: Missing values in the raw data

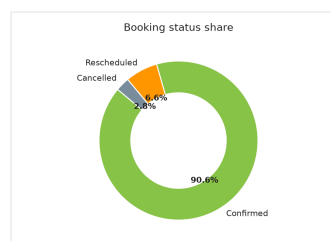


Figure 2: Share of booking status

Only three columns had gaps: duration_hours (300), airline_rating (250) and seat_availability_pct (200), a total of 750 cells, all filled with the median. 90.6% of bookings are Confirmed, 6.6% Rescheduled and 2.8% Cancelled.

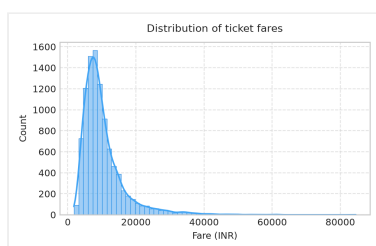


Figure 3: Distribution of ticket fares

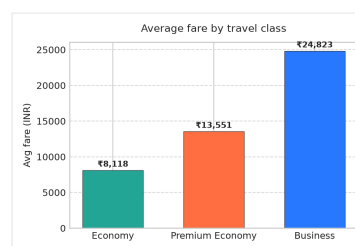


Figure 4: Average fare by travel class

Fares are strongly right-skewed: the average fare is ₹10,251 but the median is ₹8,603, and fares range from ₹1,789 to ₹84,583. The most expensive ticket is a Business-class booking on a 1,760 km route made only 3 days before departure. 64.2% of Economy fares lie between ₹3,500 and ₹9,000, while Business-class and last-minute bookings create the long tail.

Travel class is the biggest price lever: Business averages ₹24,824, Premium Economy ₹13,552 and Economy ₹8,119, so Business costs about **3.1 times** as much as Economy and Premium Economy about 1.7 times.

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DATA ANALYTICS AND DATA VISUALIZATION LAB

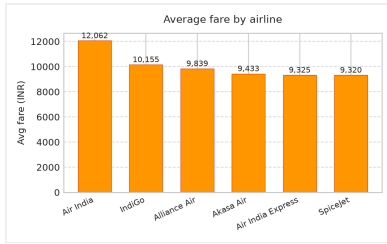


Figure 5: Average fare by airline

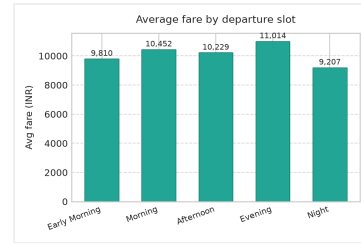


Figure 6: Average fare by departure slot

Air India is the most expensive airline (average ₹12,062), followed by IndiGo (₹10,156). SpiceJet (₹9,321), Air India Express (₹9,325) and Akasa Air (₹9,433) have the lowest averages. Evening departures are the dearest (₹11,014) and night departures the cheapest (₹9,208). Holiday-season bookings cost about 16% more (₹11,576 versus ₹9,967).

11. Project Output – Relationships and Predictors

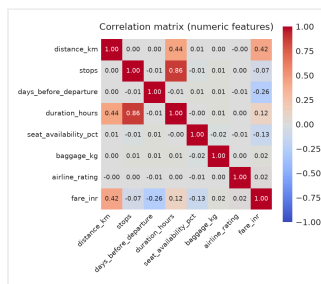


Figure 7: Correlation matrix of numeric features and fare

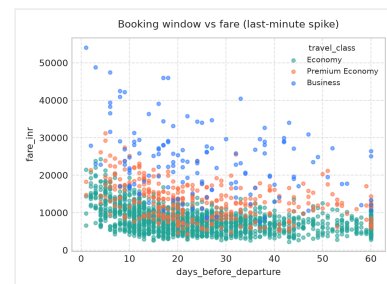


Figure 8: Booking window versus fare

Among the numeric features, route distance has the strongest linear correlation with the fare (0.42), followed by days before departure (-0.26) and seat availability (-0.13): longer routes cost more, and the earlier the booking or the more seats are free, the cheaper the ticket. Baggage allowance and airline rating are almost unrelated (0.02). Travel class is categorical, so it does not appear in the matrix although it is the strongest driver of all.

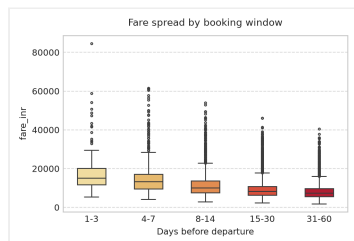


Figure 9: Fare spread by booking window

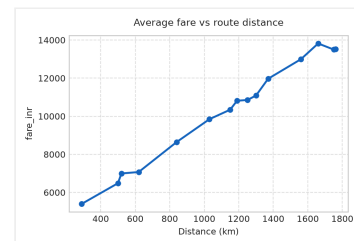


Figure 10: Average fare versus route distance

The relationship with the booking window is clearly non-linear. Tickets bought 1–3 days ahead average ₹17,584, 4–7 days ₹15,166, 8–14 days ₹11,689, 15–30 days ₹9,513 and 31–60 days only ₹8,507, so the last-minute ticket costs about 2.1 times a ticket booked a month or more ahead. Bookings made within 7 days cost 84% more on average than bookings made 31 days or more ahead, and 17.7% of all bookings are made within 10 days of departure.

Seat scarcity adds a smaller premium: flights with under 15% seats available average ₹12,681 compared with ₹10,176 for the rest. Fares also grow with distance, from about ₹5,399 on the shortest 290 km route to ₹13,524 on the longest 1,760 km route.

12. Machine Learning Models and Comparison

Two regression models were trained on the same training data and evaluated on the same unseen test records. The target is the numeric fare_inr, and the inputs are the 16 booking features (7 numeric and 9 categorical).

Models used:

- **Linear Regression** – a simple, interpretable baseline that assumes constant additive effects.
- **Random Forest Regressor** – an ensemble of 150 decision trees that captures non-linear effects and interactions.

Model	R2	RMSE (INR)	MAE (INR)
Linear Regression	0.8435	2,366.82	1,624.76
Random Forest (best)	0.9433	1,424.86	922.30

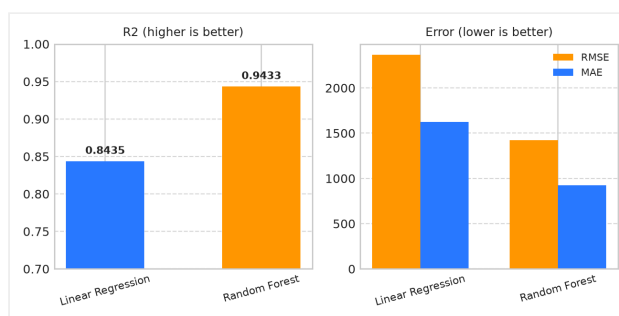


Figure 11: Model comparison

Random Forest achieved the best result with $R^2 = 0.9433$, RMSE = ₹1,424.86 and MAE = ₹922.30: on average its prediction is about ₹922 away from the real fare. This lowers the MAE by 43.2% (from ₹1,625 to ₹922) and the RMSE by 39.8% compared with Linear Regression (R^2 0.8435). Fares depend on non-linear effects such as the exponential last-minute surge and the quadratic seat-demand effect, which a straight-line model cannot capture but decision trees can.

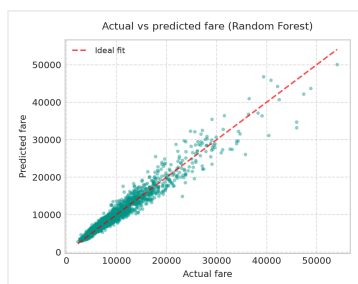


Figure 12: Actual versus predicted fare

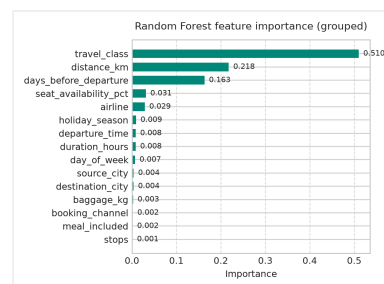


Figure 13: Random Forest feature importance

The most important predictors are travel class (51.0%), route distance (21.8%), days before departure (16.3%), seat availability (3.1%) and airline (2.9%); all other features together contribute 4.8%. This agrees with the EDA, where class, distance and booking window showed the largest price differences.

13. Web Application

The project is delivered as a Streamlit web application (app.py) with interactive Plotly charts. It has five pages:

- **Actual Dataset** – the raw 10,000 records, the 19-column data dictionary, summary metrics and a CSV download button.
- **Data Cleaning** – missing values per column before and after median imputation, duplicate check and a description of the pipeline.
- **Exploratory Analysis** – fare distribution, airline comparison, booking lead-time curves and the correlation heatmap.
- **Model Comparison** – metric cards (MAE, RMSE, R^2), actual-versus-predicted plot and feature importance.
- **Predict Fare** – a booking form (airline, route, departure slot, travel class, lead days, seat availability, baggage) that returns the predicted fare from both models.

How to run:

```
pip install -r requirements.txt
streamlit run app.py
```

The application is also deployed on Streamlit Community Cloud, and the source code is available in the GitHub repository listed above.

14. Key Findings

- The dataset has 10,000 bookings; after median imputation of 750 missing cells no record was lost.
- Travel class is the main price driver: Business costs about 3.1 times Economy and class alone accounts for about half of the model importance (51.0%).
- Last-minute bookings are the most expensive: a ticket bought 1–3 days ahead costs about 2.1 times one bought 31–60 days ahead.
- Air India is the most expensive airline (₹12,062 on average); SpiceJet, Air India Express and Akasa Air are the cheapest.
- Fewer seats and holiday periods push prices up (+25% below 15% seat availability, +16% in holiday season).

- Random Forest gives the best performance (R^2 0.9433, MAE ₹922.30) and clearly beats Linear Regression because fares are non-linear.

15. Limitations

- The dataset is synthetic: fares are produced by a formula with a fixed seed, so the patterns reflect the pricing rules built into it, not real airline data. The high R^2 partly shows that the model can rediscover these rules.
- The 750 missing values were imputed with the median, which can hide real variation.
- Only two models were compared and no hyper-parameter tuning was done; the train/test split is random, not time-based.
- Only six metro cities are covered, and holidays are a simple Yes/No flag with no real calendar or live fare feed.

16. Conclusion

The Flight Fare project demonstrates the complete DAV workflow: data handling, cleaning, exploratory analysis, visual communication, machine learning and deployment as a web application. Two models were compared on the same test data, and the best one, Random Forest ($R^2 = 0.9433$, MAE = ₹922), is used for live fare prediction. The analysis shows clearly how travel class, distance and booking lead time drive ticket prices.

The project can be extended with real-time flight pricing APIs (such as Amadeus or Skyscanner), route-specific festive surge detection with lowest-fare alerts, and gradient boosting models such as XGBoost or LightGBM for further accuracy gains.

17. References

- **Pandas:** <https://pandas.pydata.org/docs/>
- **Scikit-learn ensemble methods:** <https://scikit-learn.org/stable/modules/ensemble.html>
- **Plotly for Python:** <https://plotly.com/python/>
- **Streamlit:** <https://docs.streamlit.io/>
- **Project repository:** <https://github.com/koteswaraosabbavarapu-stack/flight-fare-analytics> (app.py, data.py, requirements.txt)